Building and Comparing Inverted Indexes on Reuters-21578

COMP 479 Project 1

Sayf AlSafadi
40271828

Overview:

I built an in-memory inverted index over the Reuters-21578 collection [2] (Distribution 1.0) in 2 ways. The first is a naive indexer, it collects every (term, document) pair, removes any duplicates, sorts them and then groups them into a postings lists. The second is a SPIMI indexer which adds DocID's straight into postings lists as it reads each article. Both simply read and keep the original Reuters NEWID values, and they both produce exactly the same index; 61,453 unique terms and 1,781,942 postings. We use two indexers because the naive version allows the steps of the build to be easily inspected, giving us a reference and the SPIMI shows the direct postings construction and work by avoiding the global sort.

Furthermore, I added 'single term' and 'AND' queries, I also measured how a series of preprocessing steps (removing numbers, case folding, stop words, stemming) changes the size of the index. In the runs that I recorded of the 10,000 pair timing test, the median built time for the naive indexer was 0.001989s and for the SPIMI it was 0.000712s. I reran these tests again, which gave me 0.002072 and 0.000749 respectively, giving us in both test cases that SPIMI was roughly 2.8x faster. During the preprocessing steps the final stemmed index had 47.4% fewer distinct terms and 50.3% fewer postings than the unfiltered one.

Introduction:

An inverted index maps each term/word to a list of documents that contain it. For instance, if you wanted to know what books in Shakespeare's collection contained the word "brutus", the indexer would be able to do that, instead of having a query scan every document.

The goal of this project is to:

1. Build a naive inverted index from term-document pairs.
2. Build the same index with a SPIMI-style method that writes directly to postings.
3. Support single-term queries and AND queries.
4. Measure how preprocessing changes the index, following the stages in Table 5.1 in the Chapter 5 of "Christopher D. Manning, Prabhakar Raghavan and Hinrich Schütze, *Introduction to Information Retrieval*, Cambridge University Press. 2008."
5. Compare the two indexers build times on the same 10,000 term-document pairs.

The Reuters collection is in 22 SGML files. The code reads these files and uses the articles original NEWID as its DocID. For each article it indexes the title then by the body, after HTML entities. If an article does not have a title or body then it is treated as empty. And any other label is simply ignored. We do this because this allows the preservation of the corpus and allows it to focus on the text rather than the metadata.

Both indexes are in memory and nonpositional, meaning a posting only says that a term appears in 'X' document and not where in the document and not how often. We do this to keep

implementation focused on, term lookup, AND queries, and comparisons.The SPIMI implementation builds posting directly as it reads and relies on increasing DocID's to keep them sorted and unique. But the SPIMI implementation isn't the full disk-based SPIMI, so it doesn't write completed index blocks, so the whole index must fit in memory.

Design and implementation:

Let's go through the pipeline of how it goes about working:

1. Reads Reuters SGML and extracts the (NEWID, title + body).
2. Split the text into tokens.
3. Turn tokens into (term, NEWID) pairs and build the inverted index.
4. Answer queries with lists of matching IDs.

Parsing and tokenization

It processes the .sgm files in a sorted order, decoded as Latin-1. Once it iterates through the document it tokenizes the text. Tokenization is best described as the breaking down of an article into the smallest discrete units of text, like words, subwords or even characters. In this case we're using an ASCII-only tokenizer, so only ASCII letters and digits are recognized and everything else separates tokens. But there are limits to an ASCII only tokenizer; Accented letters get split or dropped and punctuation can break tokens apart. For instance 6.7 becomes 6 and 7.
The preprocessing steps are removing digit-only token, lowercasing, removing stop words, and applying NLTK's porter stemmer. It's important to use the same tokenizer for both indexing and queries, so that the query terms are normalized in the same way as the index. We use regular expressions because they're simple and reproducible for the project, by sharing the tokenizer we avoid them being normalized differently from indexed terms.

Naive indexer

With respect to the naive indexer, every token it comes across, it adds a (term, DocID) pair to a list. If a term appears several times in a single article which then creates several identical pairs. The indexer then runs sorted(set(pairs)) which removes duplicates and sorts them by term (since sets don't allow duplicates), and then groups the sorted pairs into a postings list. Originally the full corpus started with 2,866,956 pairs and ended with 61,453 terms as repeated occurrences were dropped. The approach is simple and makes a great reference point for checking SPIMI, but it must hold all the pairs in memory and sort them.

SPIMI style indexer

The SPIMI indexer updates a dictionary as each token is read. It appends the current DocID to a terms postings list, if that same ID isn't already the last one in the list. The articles are read in increasing NEWID order, this allows it to skip repeats within a document and keeps the list sorted. In the timing test, the pair based version is used, which checks that the IDs never decrease, so the ordering assumption is set in place.

This whole process allows us to skip the steps of sorting and removing duplication, while It still processes every token and stores the postings into memory.

Queries

There are two types of queries that were implemented. A ‘single term’ query and an ‘AND’ query. A single term query is tokenized the same way as an index does and then gets found in the dictionary. An AND query fetches the postings list for each term and intersects them using a two pointer merge, since the lists are sorted the pointer with a smaller ID moves forward and equal IDs go into the result. In the case of more than 2 terms, lists are intersected from shortest to longest to keep intermediate results small, this keeps intermediate resultant lists smaller while preserving the AND queries result. This whole process of merging takes O(m+n) time.

In preprocessing, if it removes a term from an AND query (ex. A stop word), the query returns no documents, this behaviour is similar to all or nothing, if all terms aren’t provided then it won't provide any at all. In the cases of invalid queries (a multi-term query) sent to a single term function, it is simply rejected.

Correctness checks

To ensure the correctness of both indexes I compared them as full dictionaries, not just by their counts. They were identical, having both 61,453 terms and 1,781,942 postings being before 2,866,956 raw pairs. I also ensured that the postings were sorted and had no duplicates. It produced 21,578 articles which meant 21,578 distinct IDs in ascending order.

Furthermore, I also utilized automated tests (test_tiny.py) covering a two article example; ensuring exact postings, query hits and misses, naive/SPIMI equality, invalid queries, different tokenizer options, stop word selection, preprocessing counts, and 100 deterministic list intersections which were corrected against a pythons set intersection. (to run the test “python -m unittest test_tiny.py”)

Timing

I recorded the timing between naive and SPIMI indexers build times. In the run, SPIMI built the index ~2.79 times faster than the naive indexers median time.

I recorded the run by preparing the first 10,000 unfiltered term document pairs once, before running either index and from the 10,000 pairs there were 6,218 that are unique . The naive covers removal of duplicate terms, sorting and grouping while SPIMI covers direct dictionary updates and duplicate skipping. For both indexes, parsing and tokenization are not timed, since both indexers share the same ones. The benchmark changes depending on what indexer goes first due to caching, since the first operation already brought the data into the caches memory and the OS RAM cache which is the fastest part of hardware, so if the data is

already there to be run for the second operation, it wouldn't accurately test the difference between the two indexers. So we must alternate them and run 7 runs, checking the two indexes are consistent to each pair of runs and calculate the median.

Results

Indexer	Recorded runs Median time(sec)
Naive	0.001989
SPIMI	0.000712

Reflection

My initial thoughts match the results of these tests, since SPIMI skips the global sort and the separate deduplication. However, the timings are extremely minute, meaning they are sensitive to timer resolution and machine load. They showed us what happens in this point of reference but not as a prediction for the full corpus.

Preprocessing Experiment

Preprocessing is the pathway/pipeline to a cleaner and more standardized raw text, before that text ever reaches an index. It goes through multiple stages; first tokenization, then case folding, stop word removal and finally stemming. After the preprocessing is completed, the index has 47.7% less distinct terms, which I theorize to be heavily due to the case folding and stemming and 50.3% fewer postings which is due to the stop word removal.

Using preprocessing I created 6 different versions of the SPIMI index:

- Unfiltered
- No numbers: remove any token that is only a digit
- Case folding: lowercase every token
- 30 stop words: removes the 30 most frequent terms
- 150 stop words: removes the 150 most used terms
- Stemming: apply the porter stemmer (reduces a word to its simplest/root form ex. cats -> cat)

I figured the stop words list from the number filtered and case folded corpus, ranked by total token hits.

Results

The Change(%) is checking the unique terms(or postings) of the previous preprocessing to determine the change in its quantity of distinct terms using this formula “Percentage change = (new count − starting count) ÷ starting count × 100” but in the case of the “30” and “150” stop words they both use case folding as its reference.

Preprocessing	Unique terms	Change %	Total Change	Postings	Change %	Total Change
unfiltered	61,453	//	//	1,781,942	//	//
No numbers	59,252	-3.6%	-3.6%	1,622,359	-9.0%	-9.0%
Case folding	43,197	-27.1%	-29.7%	1,499,117	-7.6%	-15.9%
30 stop words	43,167	-0.1%	-29.8%	1,218,450	-18.7%	-31.6%
150 stop words	43,047	-0.3%	-30.0%	930,958	-37.9%	-47.8%
Stemming	32,318	-24.9%	-47.4%	886,223	-4.8%	-50.3%

Comparison with Table 5.1

The pattern in my findings match Table 5.1 [1] in the textbook, the number removal had less of an effect than case folding or stemming(which shrunk the vocab), and stop words affected postings tremendously in comparison to its effect on the vocabulary

.

Although the size of change differs between the two tables:

- Here case folding reduced the terms by 27.1% vs the 17% in table 5.1 [1] and postings being 7.6% vs 3% respectively
- The 30 and 150 stop words reduced posting by 18.7% and 37.9% respectively, Vs the 14% and 30% in the table 5.1
- Stemming here reduced the terms by 24.9% vs the 17% for table 5.1

This is to be expected since Table 5.1 uses a different corpus. The differences in corpus, tokenizers and stop lists all have a play in the difference of numbers, but the percentages are a good guideline for size of effect for each step of the preprocessing.

Queries

I picked 3 AND queries and 3 single term queries showing how preprocessing affects retrieval. For the AND queries, the matching documents contain both query terms. Each query uses the same preprocessing as its index, and the matching DocIDs are documented in the table.

Type	Query	unfiltered	No numbers	Case folding	30 stop words	150 stop words	stemming
Single	coffee	166	166	196	196	196	196
Single	trade	1605	1605	1957	1957	0	0
single	acquisition	685	685	829	829	829	1001
AND	coffee crop	25	25	29	29	29	36
AND	gold mining	39	39	89	89	89	156
AND	bankruptcy acquisition	10	10	12	12	12	13

Reflections

- For the case folding the count raises because it merges multiple terms, ex. Coffee and coffee were separate but now simply have become ‘coffee’. These multiple variations are counted and included but only counts each DocID once.
- Stemming counts are raised by merged terms. Although this improves the speed, the extra matches aren’t necessarily always interchangeable with the original word.
- Number removal didn't have any effect due to no queries having a number.
- the Query “trade” returns nothing for the 150 stop words since “trade” is one of those 150 stop words

Challenge queries

Query	Output from indexers (documents)	Naive time(s)	SPIMI time(s)
Carmark	[19758]	0.000000541	0.000000500
copper	107 docs: [22, 793, 800, 816, 1148, 1184, 1552, 1607, 2006, 2074, 2186, 2764, 2782, 2880, 3454, 3613, 3862, 4058, 4291, 4431, 4744, 4816, 5203, 5209, 5435, 5788, 5827, 5888, 6025, 6225, 6395, 6846, 6927, 7552, 7724, 7775, 8569, 8601, 9195, 11273, 11945, 12024, 12215, 12223, 12243, 12489, 12513, 12633, 12641, 12651, 12683, 12692, 12725, 12857, 12861, 12910, 12926, 12963, 12980, 12992, 13003, 13670, 13675, 13694, 13877, 13897, 14274, 14297, 14302, 14398, 14476, 14499, 14572, 14687, 14805, 14852, 15072, 15166, 15264, 15556, 15831, 15932, 15988, 16123, 16971, 17007, 17118, 17714, 18225, 18280, 18317, 18339,	0.000000500	0.000000500

	18392, 18394, 18430, 18457, 18631, 18691, 18758, 18849, 18952, 19502, 19781, 19786, 19878, 21293, 21327]		
Chrysler AND Bundesbank	null / 0	0.000013000	0.000012917

Indexer	Build time(s)
naive	3.284948
SPIMI	0.852716

References

[1] Christopher D. Manning, Prabhakar Raghavan and Hinrich Schütze, *Introduction to Information Retrieval*, Cambridge University Press. 2008. ISBN: 0521865719.
[2] Reuters-21578, Distribution 1.0 from David D. Lewis "https://archive.ics.uci.edu/dataset/137/reuters+21578+text+categorization+collection"